Assignment 7 Solution

Nicole Cruz, Julia Haaf, Anne Giacobello, Shravan Vasishth

Are subject relatives easier to process than object relatives (shifted log-normal likelihood)?

This is a classic question from the psycholinguistics literature: Are subject relatives easier to process than object relatives? The data come from Experiment 1 in a paper by Grodner and Gibson 2005.[1]

Scientific question: Is there a subject relative advantage in reading?

Grodner and Gibson 2005 investigate an old claim in psycholinguistics that object relative clause (ORC) sentences are more difficult to process than subject relative clause (SRC) sentences. One explanation for this predicted difference is that the distance between the relative clause verb (*sent* in the example below) and the head noun phrase of the relative clause (*reporter* in the example below) is longer in ORC vs. SRC. Examples are shown below. The relative clause is shown in square brackets.

(1a) The *reporter* [who the photographer *sent* to the editor] was hoping for a good story. (ORC)

(1b) The *reporter* [who *sent* the photographer to the editor] was hoping for a good story. (SRC)

The underlying explanation has to do with memory processes: Shorter linguistic dependencies are easier to process due to either reduced interference or decay, or both.

In the Grodner and Gibson data, the dependent measure is reading time at the relative clause verb, (e.g., *sent*) of different sentences with either ORC or SRC. The dependent variable is in milliseconds and was measured in a self-paced reading task. Self-paced reading is a task where subjects read a sentence or a short text word-by-word or phrase-by-phrase, pressing a button to get each word or phrase displayed; the preceding word disappears every time the button is pressed.

For this experiment, we are expecting longer reading times at the relative clause verbs of ORC sentences in comparison to the relative clause verb of SRC sentences.

```
load("df_gg05_rc.rda")
head(df_gg05_rc)
```

```
## # A tibble: 6 x 7
##    subj  item condition    RT residRT qcorrect experiment
##   <int> <int> <chr>     <int>   <dbl>    <int> <chr>
## 1     1     1 objgap      320  -21.4         0 tedrg3
## 2     1     2 subjgap     424   74.7         1 tedrg2
## 3     1     3 objgap      309  -40.3         0 tedrg3
## 4     1     4 subjgap     274  -91.2         1 tedrg2
## 5     1     5 objgap      333   -8.39        1 tedrg3
## 6     1     6 subjgap     266  -87.3         1 tedrg2
```

You should use a sum coding for the predictors. Here, object relative clauses (`"objgaps"`) are coded $+1/2$, subject relative clauses $-1/2$.

[1] Grodner, D., & Gibson, E. (2005). Consequences of the serial nature of linguistic input for sentenial complexity. Cognitive Science, 29(2), 261-290.

```r
df_gg05_rc <- df_gg05_rc %>%
  mutate(c_cond = if_else(condition == "objgap", 1/2, -1/2))
```

Before fitting, also clean the reading times: exclude relative clause verb reading times below 150 ms, since responses that fast probably are an anticipatory button-press instead of actually reading the target word and reacting. Report how many trials this excludes.

```r
n_before <- nrow(df_gg05_rc)
df_gg05_rc <- df_gg05_rc %>% filter(RT >= 150)
n_before - nrow(df_gg05_rc)
```

```
## [1] 4
```

Reading times can never be shorter than some minimum time (the time needed for basic perceptual other processes necessary for reading). We can capture this by shifting the log-normal distribution away from zero by an additional parameter (`ndt`, non-decision time). You can now fit a "maximal" model (correlated varying intercept and slopes for subjects and for items) assuming a **shifted log-normal likelihood**.

(a) Specify the statistical model and define priors for all parameters in the model, including the additional shift parameter `ndt`. *Hint:* To get a feel for reasonable ranges of values for the μ and σ parameters of a (shifted) log-normal distribution, you might want to simulate log-normal data for different values using `rlnorm()` and then add a constant shift to it. Also think about what constraints `ndt` needs to satisfy relative to the observed data for the model to be identifiable, and where the bound for its prior should come from. Should it be extracted from this particular sample, or fixed in advance?

Caution: `brms` uses a `log` link for `ndt` by default (check `?brms::shifted_lognormal`), which means that, depending on how you specify the model, the units your prior and posterior for `ndt` are expressed in may *not* be milliseconds. Before interpreting or setting a prior for `ndt`, check `make_stancode()` for your model to see exactly how `ndt` is parametrized.

(b) Examine the effect of relative clause attachment site (the predictor `c_cond`) on reading times `RT` (β_1).

(c) Estimate the median difference between relative clause attachment sites in milliseconds, and report the mean and 95% CI. Remember to take the estimated shift `ndt` into account when you transform back to the millisecond scale.

(d) Inspect individual differences in the subject relative advantage effect. Are there participants with effects in opposite directions?

(e) Do a sensitivity analysis. What is the estimate of the effect (β_1) under different priors? What is the difference in milliseconds between conditions under different priors? Does the choice of prior on `ndt` also affect the results?

(f) So far `ndt` has been a single constant shared by everyone. In reality, people probably differ in their non-decision time too. Extend the model so that `ndt` varies by subject (`ndt ~ 1 + (1 | subj)`), and think carefully about what prior to put on it. Keep in mind the `log` link caution from part (a). *Hint:* a prior with mean around 5.5 to 6.5 on the *linear predictor* scale for `ndt`'s intercept, together with a weakly-informative prior on its between-subject SD, could work out well here. Think about why, and what those numbers correspond to once exponentiated back to milliseconds. You also need to to give the sampler some sensible starting values.

Solution

(a)

A shifted log-normal distribution means here that $Y - ndt$ (not Y) is log-normally distributed, where $ndt > 0$ is a constant shift (the non-decision time). Basically: $Y = ndt + \exp(N(\mu, \sigma))$. Each observation comes from one subject i reading one item k. Every subject reads every item exactly once in this design, the pair (i, k) already identifies the observation, so we don't need a separate index for the trial. We fit:

$$Y_{ik} \sim \text{ShiftedLogNormal}(\mu_{ik}, \sigma, ndt) \tag{1}$$

with

$$\mu_{ik} = \beta_0 + \delta_{i,0} + \omega_{k,0} + X_{ik} \cdot (\beta_1 + \delta_{i,1} + \omega_{k,1}) \tag{2}$$

What each term is doing:

- Y_{ik}: the observed reading time for subject i on item k
- μ_{ik}: the model's predicted mean for that specific subject-item combination
- β_0: the population-average log reading time at $X_{ik} = 0$, i.e. mean across both conditions (sum coding is centered at zero)
- X_{ik}: the sum-coded condition for that observation ($+1/2$ objgap, $-1/2$ subjgap)
- β_1: the population-average condition effect, β_1 is the average log-scale difference between the two conditions
- $\delta_{i,0}$: subject i's deviation from β_0, so how much faster/slower that person reads overall, regardless of condition
- $\delta_{i,1}$: subject i's deviation from β_1, so how much bigger/smaller that person's condition effect is compared to the population average
- $\omega_{k,0}$, $\omega_{k,1}$: the same idea for item k, some sentences are read faster/slower overall ($\omega_{k,0}$), and some show a bigger or smaller than average condition effect ($\omega_{k,1}$)

So: population intercept and slope, adjusted for this subject and this item, and combined with the condition to get the predicted mean.

Note that μ and σ describe the distribution of $Y - ndt$, and not of Y. This is because part of the reading time is "used up" by the shift. $\exp(\mu)$ is therefore a little smaller than it would be in "just" a log-normal model, so the intercept prior needs to be shifted down a bit accordingly.

- Priors:

$$\begin{aligned} \beta_0 &\sim \text{Normal}(5, 0.7) \\ \beta_1 &\sim \text{Normal}(0, 0.1) \\ \sigma &\sim \text{Normal}_+(0, 1) \\ ndt &\sim \text{Uniform}(0, 150\text{ms}) \end{aligned} \tag{3}$$

The newest here is the prior on `ndt`. Because $Y = ndt + \exp(N(\mu, \sigma))$ and $\exp(N(\mu, \sigma)) > 0$, the model is only well defined and identifiable if ndt is smaller than the smallest observed reading time. One choice could be $ndt \sim \text{Uniform}(0, \min(Y))$ using the sample minimum, this would make the prior depend on the data it is meant to be independent of, which can raise eyebrows (and could be called double-dipping). We already excluded trials below 150 ms when cleaning the data, so we know before looking at anything in the sample, that τ cannot exceed this cut-off. We therefore use a fixed bound:

As before, we need priors for the by-subject and by-item variance-covariance matrices $\mathbf{\Sigma_\delta}$ and $\mathbf{\Sigma_\omega}$:

$$
\begin{aligned}
\begin{pmatrix} \delta_{i,0} \\ \delta_{i,1} \end{pmatrix} &\sim \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \boldsymbol{\Sigma_\delta} \right) \\
\begin{pmatrix} \omega_{k,0} \\ \omega_{k,1} \end{pmatrix} &\sim \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \boldsymbol{\Sigma_\omega} \right)
\end{aligned} \tag{4}
$$

$$
\begin{aligned}
\sigma_{\delta_0} &\sim Normal_+(0, 1) \\
\sigma_{\delta_1} &\sim Normal_+(0, 1) \\
\rho_\delta &\sim LKJcorr(2) \\
\sigma_{\omega_0} &\sim Normal_+(0, 1) \\
\sigma_{\omega_1} &\sim Normal_+(0, 1) \\
\rho_\omega &\sim LKJcorr(2)
\end{aligned} \tag{5}
$$

Note Again, in `brms`, the shifted log-normal family is defined with default links `mu = identity`, `sigma = log`, and **`ndt = log`** (see `?brms::shifted_lognormal` or `brms:::shifted_lognormal`). If you let `ndt` be predicted by a formula (e.g. `ndt ~ 1 + (1 | subj)`, which you might want to do in part (f) to look at individual differences in non-decision time), the `ndt`-related coefficients live on the **log** scale, not on the millisecond scale of `RT`, so you have to `exp()` them. The priors then also need to be specified on that log scale, not directly in milliseconds.

Our model here sidesteps this but only because we don't give `ndt` a formula (it's a constant) and we define `lb`/`ub` bounds in the `prior()` call. When we do that, `brms` declares `ndt` on the natural (millisecond) scale with a `<lower=0,upper=150>` constraint. We can confirm this by inspecting the generated Stan code:

```
cat(grep("ndt", strsplit(make_stancode(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
  family = shifted_lognormal(),
  prior = c(
    prior(normal(5, 0.7), class = Intercept),
    prior(normal(0, .1), class = b),
    prior(uniform(0, 150), class = ndt, lb = 0, ub = 150)
  ),
  data = df_gg05_rc
), "\n")[[1]], value = TRUE), sep = "\n")
```

```
## real<lower=0,upper=150> ndt;  // non-decision time parameter
```

You should see a line declaring `real<lower=0,upper=150> ndt;` in the `parameters` block. This would confirm that in this model `ndt` is on the RT scale, and that `posterior_summary(fit_shifted, "ndt")` can be read in milliseconds. If you would extend this model so that `ndt` varies by subject or item (as in part (f)) you can re-run this check before you trust the scale of the `ndt`-related estimate or prior.

As a sanity check, we confirm that no remaining reading time is below our 150 ms exclusion cutoff:

```
min(df_gg05_rc$RT)
```

```
## [1] 155
```

Also, we have to check whether `brms` might turn `c_cond` into a dummy-coded factor because it only takes two values. But `brms` only automatically does factor-to-dummy-coding when it's a `character`/`factor` column, not when it's a numeric one:

```r
class(df_gg05_rc$c_cond)
```

```
## [1] "numeric"
```

```r
sort(unique(df_gg05_rc$c_cond))
```

```
## [1] -0.5  0.5
```

We can look at the design matrix `brms` hands into Stan:

```r
sd_check <- make_standata(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
  data = df_gg05_rc,
  family = shifted_lognormal()
)
sort(unique(sd_check$X[, "c_cond"]))
```

```
## [1] -0.5  0.5
```

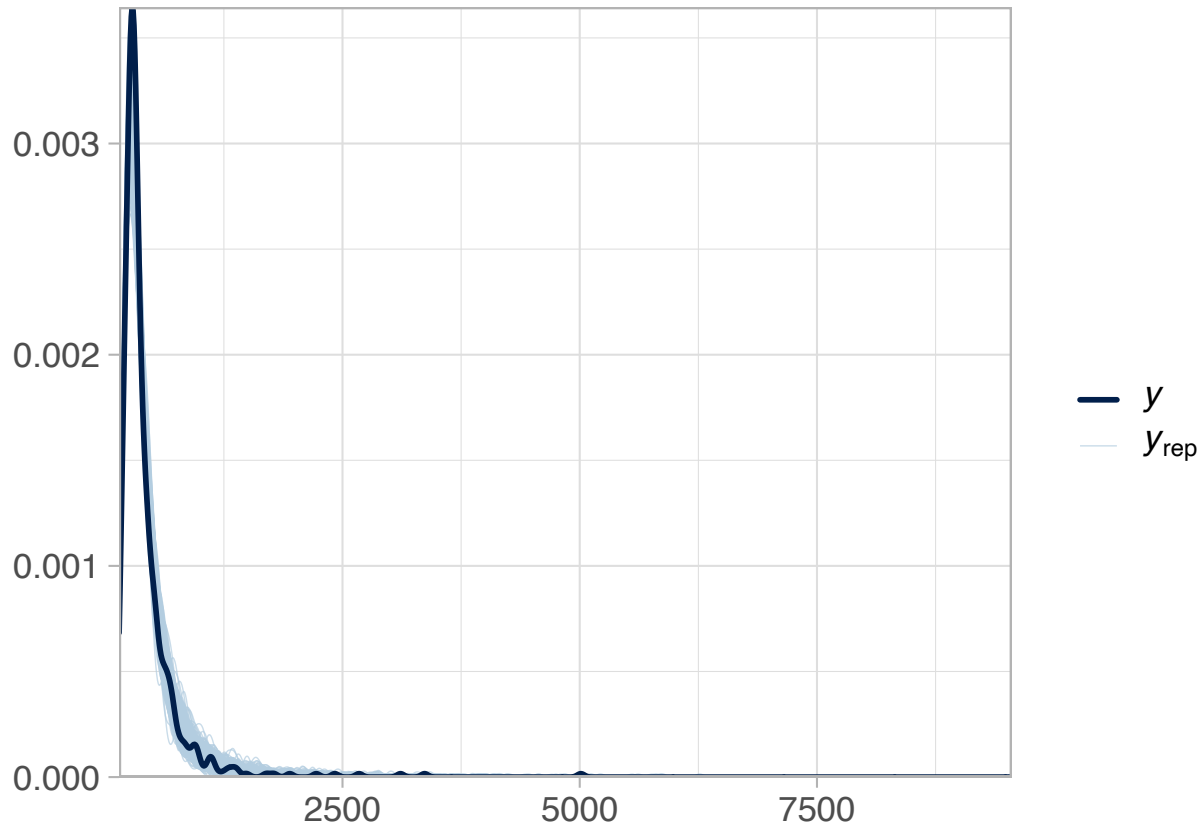
Both confirm the values going in are -0.5 and $+0.5$, not 0/1, so the sum coding stays. However, if we would put the `condition` character column into the formula, this would have triggered `brms` to do default dummy coding, and β_1 would then mean something else.

Now we can fit the model.

```r
fit_shifted <- brm(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
  family = shifted_lognormal(),
  iter = 10000,
  prior =
    c(
      prior(normal(5, 0.7), class = Intercept),
      prior(normal(0, .1), class = b),
      prior(normal(0, 1), class = sigma),
      prior(normal(0, 1), class = sd),
      prior(lkj(2), class = cor),
      prior(uniform(0, 150), class = ndt, lb = 0, ub = 150)
    ),
  data = df_gg05_rc,
  control = list(adapt_delta = 0.99, max_treedepth = 15)
)
```

```r
posterior.predictions <- posterior_predict(fit_shifted)

bayesplot::ppc_dens_overlay(df_gg05_rc$RT,
                            posterior.predictions[1:500, ]) +
  theme_light(base_size = 16)
```




```
fit_prior <- brm(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
  family = shifted_lognormal(),
  prior =
    c(
      prior(normal(5, 0.7), class = Intercept),
      prior(normal(0, .1), class = b),
      prior(normal(0, 1), class = sigma),
      prior(normal(0, 1), class = sd),
      prior(lkj(2), class = cor),
      prior(uniform(0, 150), class = ndt, lb = 0, ub = 150)
    ),
  data = df_gg05_rc,
  sample_prior = "only",
  control = list(adapt_delta = 0.99, max_treedepth = 15)
)
```

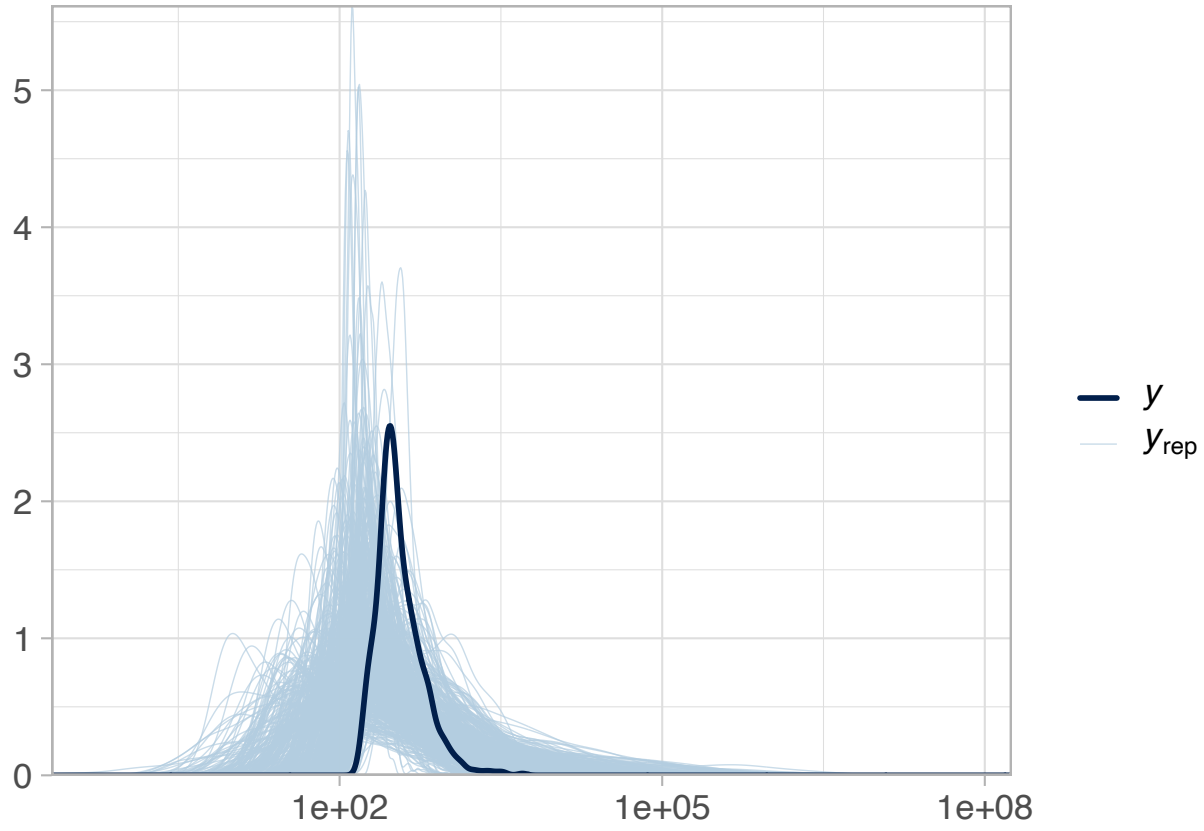
```
## Compiling Stan program...

## Trying to compile a simple C file

## Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
## using C compiler: 'Apple clang version 14.0.3 (clang-1403.0.22.14.1)'
## using SDK: 'MacOSX13.3.sdk'
## clang -arch arm64 -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG   -I"/Library/Frame
```

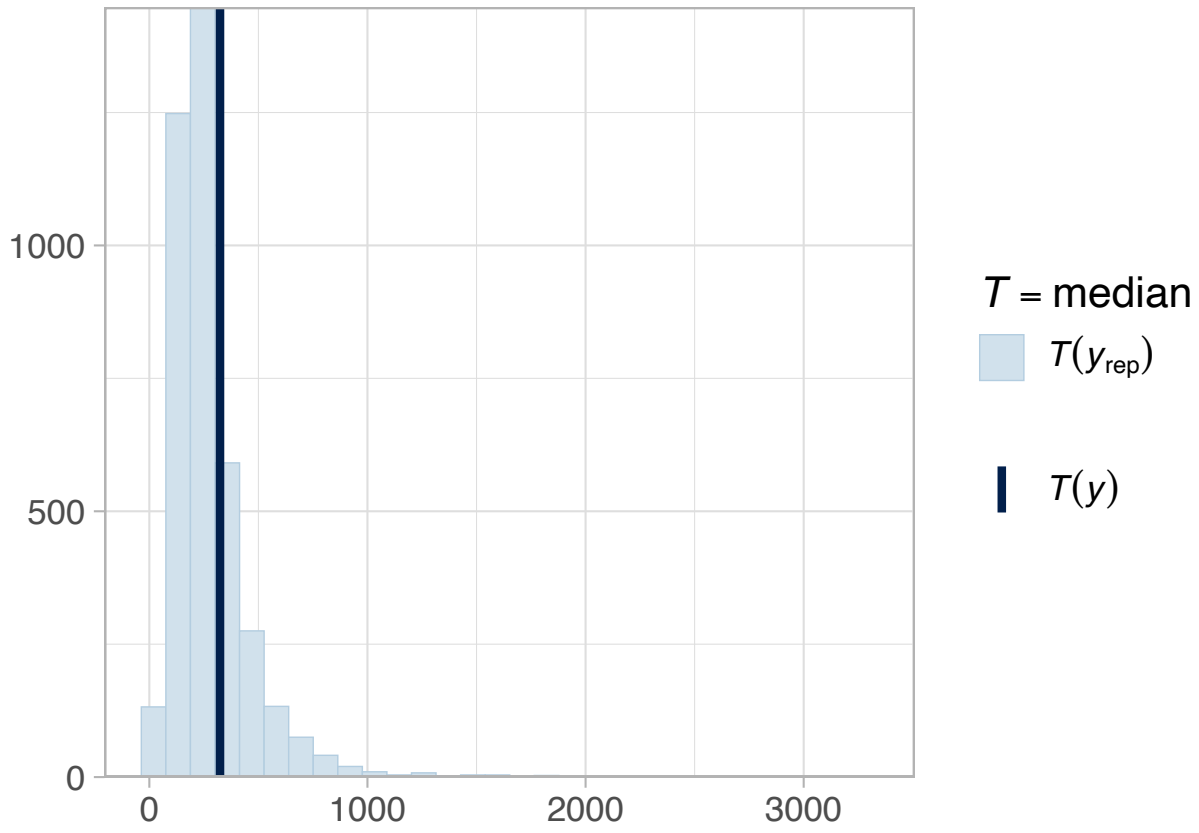
```
## Chain 4:  Elapsed Time: 0.206 seconds (Warm-up)
## Chain 4:                0.151 seconds (Sampling)
## Chain 4:                0.357 seconds (Total)
## Chain 4:
```

```
prior.predictions <- posterior_predict(fit_prior)
bayesplot::ppc_dens_overlay(df_gg05_rc$RT,
                            prior.predictions[1:500, ]) +
  scale_x_log10() +
  theme_light(base_size = 16)
```




```
bayesplot::ppc_stat(df_gg05_rc$RT, prior.predictions, stat = "median") +
  theme_light(base_size = 16)
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



sigma ~ Normal+(0,1) PLUS with Intercept ~ Normal(5, 0.7) means occasional very big RTs (a half-normal(0,1) prior on sigma has mass out past 2–3, and on a log scale that's enough to produce simulated RTs that are way too large)

(b) Examine the effect of relative clause attachment site (the predictor `c_cond`) on reading times `RT` in log-scale.

We'll focus on β_1, this is unaffected by the shift:

```
posterior_summary(fit_shifted, variable = "b_c_cond")
```

```
##            Estimate  Est.Error        Q2.5     Q97.5
## b_c_cond 0.09345305 0.06640295 -0.03908915 0.2219332
```

We can also check the posterior for `ndt`, and check whether it stays below the observed minimum. Because we did not give `ndt` a formula, this estimate is already on the millisecond scale:

```
posterior_summary(fit_shifted, variable = "ndt")
```

```
##     Estimate Est.Error     Q2.5    Q97.5
## ndt  146.592   2.26802 141.2593 149.7587
```

```r
min(df_gg05_rc$RT)
```

```
## [1] 155
```

(c)

For a shifted log-normal, the expected reading time in condition X is $ndt + \exp(\beta_0 + \beta_1 X)$. Because `ndt` is a constant additive shift that is the same for both conditions, it cancels out when we compute the difference between conditions, so the difference formula is the same as if we would have a log-normal model:

```r
draws <- as_draws_df(fit_shifted)
beta0 <- draws$b_Intercept
beta1 <- draws$b_c_cond
ndt   <- draws$ndt

# difference between object RC (coded .5) and subject RC (coded -.5):
# the ndt term cancels out of the difference
effect <- exp(beta0 + beta1 * .5) - exp(beta0 + beta1 * -.5)
c(mean = mean(effect), quantile(effect, c(.025, .975)))
```

```
##      mean      2.5%     97.5%
## 17.091174 -6.698166 42.615859
```

It is still useful to report the condition-specific predicted reading times. Here, `ndt` does not cancel out:

```r
rt_objgap <- ndt + exp(beta0 + beta1 * .5)
rt_subjgap <- ndt + exp(beta0 + beta1 * -.5)
c(mean_objgap = mean(rt_objgap), mean_subjgap = mean(rt_subjgap))
```

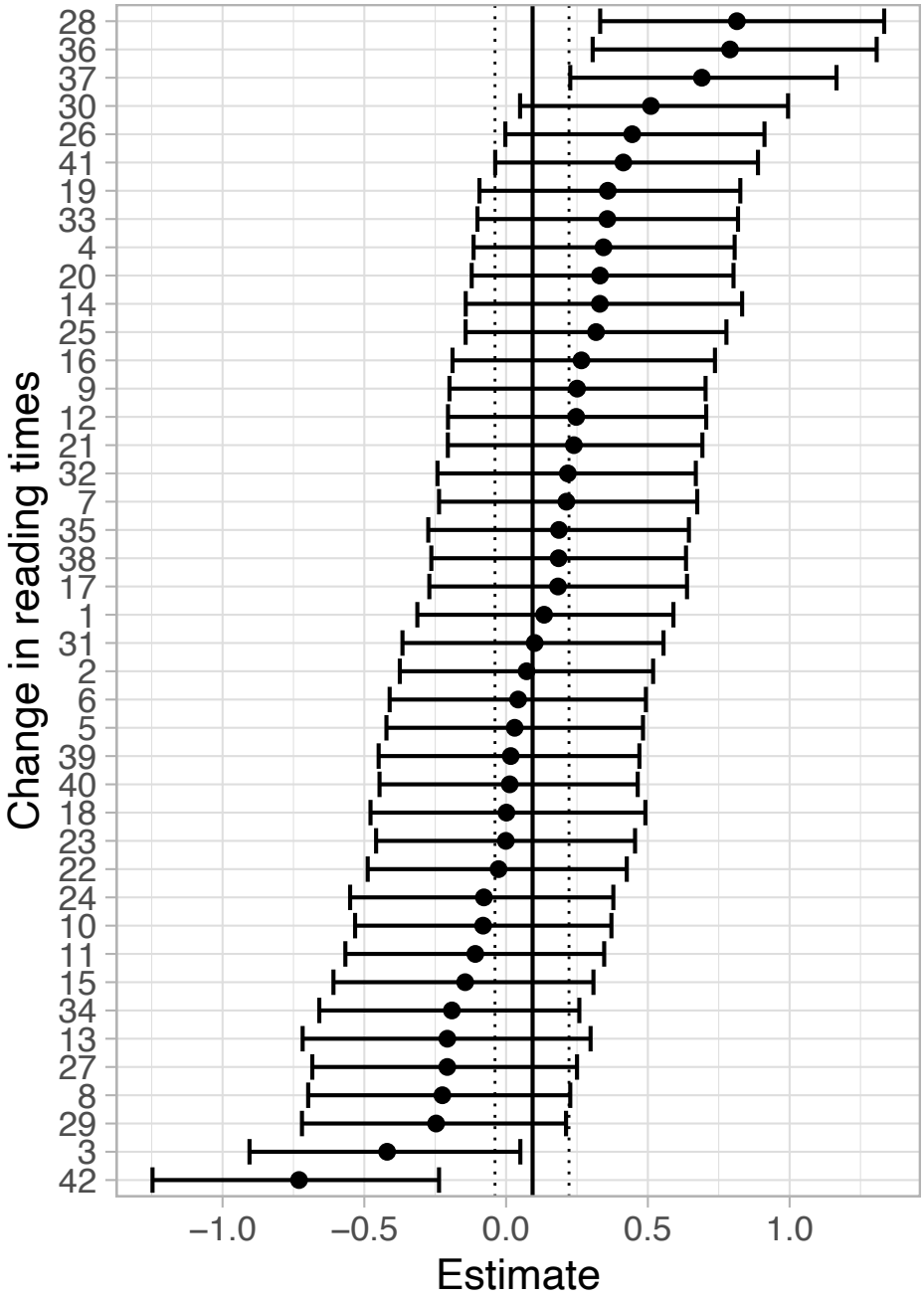
```
##  mean_objgap mean_subjgap
##     335.4942     318.4030
```

(d)

```r
# We get the by-subject effects in a data frame
by_subj_effect <- data.frame(coef(fit_shifted)$subj[,, "c_cond"]) %>%
  bind_rows() %>%
  # We add a column with the subject labels
  mutate(
    subj = unique(df_gg05_rc$subj)) %>%
  arrange(Estimate) %>%
  mutate(subj = factor(subj, levels = subj))

ggplot(
  by_subj_effect,
  aes(
    ymin = Q2.5, ymax = Q97.5, x = subj, y = Estimate
  )
) +
```

```
  geom_errorbar() +
  geom_point() +
  # We also add the mean and 95% CrI of the overall difference to the plot
  geom_hline(
    yintercept =
      posterior_summary(fit_shifted)["b_c_cond", "Estimate"]
  ) +
  geom_hline(
    yintercept =
      posterior_summary(fit_shifted)["b_c_cond", "Q2.5"],
    linetype = "dotted", linewidth = 0.5
  ) +
  geom_hline(
    yintercept =
      posterior_summary(fit_shifted)["b_c_cond", "Q97.5"],
    linetype = "dotted", linewidth = 0.5
  ) +
  xlab("Change in reading times") +
  coord_flip() +
  theme_light(base_size = 16)
```



As before, some subjects' credible intervals cross zero or point in the direction opposite to the group-level estimate, showing that not everyone shows the same (or even same-signed) relative advantage.

(e) Sensitivity analysis

We first vary the prior on β_1, keeping the `ndt` prior fixed.

Tighter prior, $\beta_1 \sim Normal(0, 0.01)$:

```
fit_shifted2 <- brm(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
```

```
  family = shifted_lognormal(),
  prior =
    c(
      prior(normal(5, 0.7), class = Intercept),
      prior(normal(0, 0.01), class = b),
      prior(normal(0, 1), class = sigma),
      prior(normal(0, 1), class = sd),
      prior(lkj(2), class = cor),
      prior(uniform(0, 150), class = ndt, lb = 0, ub = 150)
    ),
  data = df_gg05_rc,
  control = list(adapt_delta = 0.99, max_treedepth = 15)
)
```

Wider prior, $\beta_1 \sim Normal(0, 1)$:

```
fit_shifted3 <- brm(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
  family = shifted_lognormal(),
  prior =
    c(
      prior(normal(5, 0.7), class = Intercept),
      prior(normal(0, 1), class = b),
      prior(normal(0, 1), class = sigma),
      prior(normal(0, 1), class = sd),
      prior(lkj(2), class = cor),
      prior(uniform(0, 150), class = ndt, lb = 0, ub = 150)
    ),
  data = df_gg05_rc,
  control = list(adapt_delta = 0.99, max_treedepth = 15)
)
```

```
## Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and medians may be
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess
```

Comparing β_1 across priors:

Estimate	Q2.5	Q97.5	Prior for β_1
0.0020108	-0.0176566	0.0212179	Normal(0,.01)
0.0934530	-0.0390892	0.2219332	Normal(0,.1)
0.1635331	-0.0095757	0.3394224	Normal(0,1)

And the corresponding difference in milliseconds (`ndt` cancels out, see (c)):

Estimate (ms)	Q2.5	Q97.5	Prior for β_1
0.3567226	-3.054180	3.841477	Normal(0,.01)
17.0911737	-6.698166	42.615859	Normal(0,.1)
30.3551520	-1.521847	65.867409	Normal(0,1)

As in a log-normal model, the posterior for β_1 and the (implied) millisecond difference change a lot depending on how informative the prior is, especially when the effect is small relative to the data.

Finally, we check whether the choice of prior on `ndt` matters. We compare the $ndt \sim \text{Uniform}(0, 150\text{ms})$ against a more informative choice that concentrates mass closer to zero, e.g. a half-normal truncated at 150 ms, $ndt \sim \text{Normal}_+(0, 100)$ restricted to $(0, 150)$:

```
fit_shifted_ndt <- brm(
  RT ~ c_cond + (c_cond | subj) + (c_cond | item),
  family = shifted_lognormal(),
  prior =
    c(
      prior(normal(5, 0.7), class = Intercept),
      prior(normal(0, .1), class = b),
      prior(normal(0, 1), class = sigma),
      prior(normal(0, 1), class = sd),
      prior(lkj(2), class = cor),
      prior(normal(0, 100), class = ndt, lb = 0, ub = 150)
    ),
  data = df_gg05_rc,
  control = list(adapt_delta = 0.99, max_treedepth = 15)
)
```

ndt Estimate	beta1 Estimate	Diff (ms)	Prior for ndt
146.592	0.0934530	17.09117	Uniform(0, 150ms)
146.504	0.0956079	17.62653	Normal+(0,100)

Because `ndt` cancels out of the difference between conditions, changing its prior mostly shifts the estimate of `ndt` (and also β_0/σ), but has little effect on β_1 or on the millisecond difference between conditions.

f) Individual differences in `ndt`

Instead of a single `ndt`, we now let it vary by subject. But now `ndt` gets a formula, so the log-link note from part (a) becomes relevant: `ndt`'s intercept and by-subject SD now live on the log scale, not milliseconds.

```
bprior_ndt <- c(
  prior(normal(5, 0.7), class = Intercept),
  prior(normal(0, .1), class = b),
  prior(normal(0, 1), class = sigma),
  prior(normal(0, 1), class = sd),
  prior(lkj(2), class = cor),
  prior(normal(6, 0.5), class = Intercept, dpar = ndt, lb = 0),
  prior(exponential(1), class = sd, dpar = ndt)
)

fit_shifted_indiv_ndt <- brm(
  bf(RT ~ c_cond + (c_cond | subj) + (c_cond | item),
     ndt ~ 1 + (1 | subj)),
  family = shifted_lognormal(),
  prior = bprior_ndt,
  data = df_gg05_rc,
```

```
  init = 0.1,
  control = list(adapt_delta = 0.99, max_treedepth = 15)
)
```

A few things worth noting about this prior:

- `normal(6, 0.5)` on `ndt`'s intercept looks like it implies a very tiny non-decision time, but it doesn't because of the log link. `6` is on the linear predictor scale, and the corresponding non-decision time is $\exp(6) \approx 403$ ms, so a plausible value for reading. The SD of 0.5 means the bulk of the prior mass for an individual subject falls somewhere between $\exp(5) \approx 148$ and $\exp(7) \approx 1097$ ms.
- `lb = 0` bounds the linear predictor, not `ndt`. Since `ndt` is already positive after exponentiating, this barely has an effect, it only rules out non-decision times below $\exp(0) = 1$ ms. It is not the same kind of hard bound we used for the fixed-`ndt` model.
- The by-subject SD gets a weakly informative `exponential(1)` prior, this is a common prior for a group-level SD on a log scale.
- Here is no simple way to give each subject's `ndt` a per-subject upper bound tied to their own minimum RT. Identifiability relies on the prior being concentrated well below the observed RTs, so implausible regions get basically no prior mass.
- A badly-chosen starting point can effect the sampler to sample in an invalid region (where $Y_{ik} - ndt_i < 0$ for some subject), so it helps to set starting values, e.g. `init = 0.1`, instead of leaving `init` at its default.